

CURSO DE C++

CAPÍTULO 4

Tipos de Datos en C++

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C++

Capítulo técnico, educativo y profesional

Fundamentos, tipos fundamentales, memoria, rango, precisión, conversiones, tipos compuestos, buenas prácticas, ejercicios y proyecto.

Objetivo del capítulo

Aprenderás qué es un tipo de dato, por qué C++ distingue entre números, caracteres, texto y valores lógicos, y cómo elegir el tipo adecuado para cada variable. El capítulo avanza desde los tipos fundamentales hasta conceptos prácticos como **sizeof**, **numeric_limits**, conversiones, **const**, **auto**, **enum class** y **struct**.

Al finalizar este capítulo serás capaz de:

- Definir qué es un tipo de dato y explicar su función.
- Diferenciar int, short, long, long long, unsigned, float, double, char, bool y std::string.
- Elegir tipos según significado, rango, precisión y requisitos del sistema.
- Consultar tamaños con sizeof y límites con numeric_limits.
- Comprender conversiones implícitas y explícitas.
- Detectar pérdida de información y problemas de rango.
- Utilizar const y auto correctamente.
- Representar estados con enum class y entidades con struct.
- Aplicar tipos en sistemas de máquinas, sensores, producción e inventarios.
- Diseñar un pequeño modelo de datos para una estación de producción.

Introducción al tema

¿Qué es un tipo de dato?

Un tipo de dato es una clasificación que indica qué clase de valor puede representar una variable y qué reglas se aplican a ese valor. Por ejemplo, **int** representa números enteros y **double** representa números de punto flotante.

Analogía

Imagina un almacén con recipientes etiquetados. Un recipiente para líquidos no se maneja igual que uno para piezas pequeñas. La etiqueta indica cómo interpretar su contenido. En C++, el tipo cumple una función parecida: le comunica al compilador y al programador qué clase de información se está manejando.

¿Por qué existe?

El hardware trabaja con bits y memoria, pero el software necesita representar conceptos como edad, temperatura, piezas, nombres, estados y mediciones. Los tipos permiten expresar esos conceptos de forma estructurada.

¿Qué problema resuelve?

- Organiza la representación de datos.
- Permite operaciones apropiadas.
- Ayuda al compilador a detectar usos incompatibles.
- Permite razonar sobre memoria, rango y precisión.
- Hace el código más expresivo y mantenible.

Importancia real

En automatización, una temperatura puede necesitar decimales; una cantidad de piezas normalmente es entera; el estado de una máquina puede ser booleano o uno de varios estados; el código de una máquina puede ser texto aunque contenga

números. Elegir mal el tipo puede causar pérdida de información o lógica incorrecta.

Relación con el curso

Las variables necesitan tipos; las entradas se almacenan en variables; las salidas muestran sus valores; los comentarios pueden documentar unidades y decisiones. Los tipos también serán fundamentales para operadores, decisiones, arrays, funciones, POO y estructuras de datos.

Explicación completa y progresiva

1. Variable, tipo y valor

```
int piezas = 500;
^          ^
tipo       valor

piezas
^
nombre de la variable
```

La variable se llama **piezas**, su tipo es **int** y comienza con el valor 500. El tipo influye en cómo se representa el valor y en qué operaciones son apropiadas.

2. Bits y bytes

Un bit representa dos estados. Un byte es una unidad de almacenamiento definida por C++ a través de CHAR_BIT; en las plataformas modernas habituales son 8 bits. La cantidad de combinaciones depende del número de bits.

```
8 bits -> 2^8 = 256 combinaciones
16 bits -> 2^16 = 65 536 combinaciones
32 bits -> 2^32 = 4 294 967 296 combinaciones
```

Estas combinaciones no significan automáticamente que un tipo con N bits tenga exactamente ese rango de enteros con signo: la representación y las reglas del tipo importan.

Tipos fundamentales y tipos esenciales

int — números enteros

Qué es: un entero. **Por qué:** muchas cantidades son discretas. **Cuándo:** piezas, empleados, contadores. **Ejemplo:**

```
int piezas = 500;
int empleados = 35;
int contador = 0;
```

No es apropiado usar int para una medida que realmente necesita una fracción, como 25.7 °C.

short, long y long long

Son variantes enteras con diferentes garantías mínimas y tamaños dependientes de la implementación. No debes asumir que sus tamaños son iguales en todos los sistemas.

```
short muestras = 100;
long distancia = 100000L;
long long produccionTotal = 5000000000LL;
```

Elige estas variantes cuando el rango, la memoria, una interfaz o un requisito de plataforma lo justifique.

unsigned

Un entero unsigned representa valores no negativos. Puede ser útil cuando el dominio realmente es no negativo, pero mezclarlo con enteros con signo requiere comprender las conversiones.

```
unsigned int stock = 500;
```

```
unsigned int piezas = 0;
```

No significa automáticamente que sea una opción mejor o más segura.

char

Representa un carácter y también es un tipo entero pequeño. Para un carácter individual se usan comillas simples.

```
char inicial = 'F';
char codigo = 'A';
```

bool

Representa un valor lógico: **true** o **false**. Es apropiado para estados binarios.

```
bool maquinaActiva = true;
bool alarma = false;
```

float y double

Representan números de punto flotante. **double** suele ofrecer más precisión que **float** en las implementaciones habituales, pero debes revisar requisitos concretos.

```
float temperaturaRapida = 25.5f;
double temperatura = 25.5;
```

El sufijo f indica un literal float; un literal decimal sin ese sufijo es normalmente double.

void

Indica ausencia de valor en determinados contextos, especialmente en funciones que no devuelven un resultado.

```
void mostrarMensaje() {
    // realiza una acción
}
```

std::string

Es una clase de la biblioteca estándar para manejar texto de longitud variable. Es esencial en aplicaciones reales.

```
#include <string>
std::string maquina = "Torno CNC";
std::string modelo = "SE34";
```

Tabla de referencia

Tipo	Representa	Ejemplo	Uso típico
bool	Lógico	true	Estados binarios
char	Carácter	'A'	Carácter individual
int	Entero	500	Contadores y cantidades
short	Entero	100	Cuando el rango requerido lo justifica
long	Entero	100000L	Valores grandes según plataforma
long long	Entero grande	5000000000LL	Contadores grandes
float	Punto flotante	25.5f	Memoria/precisión según requisito
double	Punto flotante	25.5	Cálculo decimal general
void	Sin valor	void f()	Funciones sin retorno
std::string	Texto	"Torno"	Nombres y mensajes

Tamaño, rango y precisión

sizeof

sizeof permite consultar cuántos bytes ocupa un tipo u objeto en la implementación concreta.

```
#include <iostream>

int main() {
    std::cout << sizeof(int) << std::endl;
    std::cout << sizeof(double) << std::endl;
}
```

El resultado puede variar. Si el tamaño es un requisito de portabilidad, no lo supongas: consulta el sistema o utiliza tipos con propiedades adecuadas.

numeric_limits

La biblioteca **<limits>** permite consultar propiedades numéricas.

```
#include <iostream>
#include <limits>

int main() {
    std::cout << std::numeric_limits<int>::min() << std::endl;
    std::cout << std::numeric_limits<int>::max() << std::endl;
}
```

Rango

Es el conjunto de valores que el tipo puede representar. Diseñar pensando en el rango es importante cuando un contador puede crecer mucho o una medida puede superar valores normales.

Precisión

Es la capacidad de conservar detalles significativos. Los tipos de punto flotante no representan todos los números decimales exactamente; por ello, algunas comparaciones requieren tolerancia.

Overflow

Es intentar representar un resultado fuera del rango. En código profesional se deben analizar límites, entradas y operaciones antes de confiar en que un valor siempre cabrá.

Punto flotante

float y double suelen basarse en representaciones de punto flotante y muchas plataformas utilizan IEEE 754. C++ no debe tratarse como si obligara a todas las implementaciones a idénticos detalles en cualquier plataforma.

Conversiones de tipos

Conversión implícita

```
int piezas = 100;
double total = piezas; // conversión automática
```

C++ puede convertir automáticamente un valor cuando las reglas del lenguaje lo permiten.

Conversión explícita con static_cast

```
double temperatura = 25.8;
int entero = static_cast<int>(temperatura);
```

El resultado es 25 porque la conversión a entero descarta la fracción. No es un redondeo.

Pérdida de información

```
double precio = 19.99;
int valor = static_cast<int>(precio); // 19
```

Antes de convertir debes decidir si perder la fracción es aceptable.

Conversión y entradas

El tipo de la variable también determina qué clase de dato espera **cin**. Introducir texto cuando se espera un número puede dejar el flujo de entrada en estado de error.

Conceptos adicionales importantes

const

const indica que un objeto no puede modificarse a través de ese nombre después de su inicialización.

```
const double LIMITE_TEMPERATURA = 80.0;
```

auto

auto permite deducir el tipo a partir del inicializador. No elimina el tipado estático.

```
auto piezas = 500;           // int
auto temperatura = 25.5;     // double
```

enum class

Representa un conjunto de opciones nombradas. Es especialmente útil para estados mutuamente excluyentes.

```
enum class EstadoMaquina {
    Detenida,
    Produciendo,
    Mantenimiento
};

EstadoMaquina estado = EstadoMaquina::Produciendo;
```

struct

Permite agrupar datos relacionados en un tipo compuesto.

```
struct Sensor {
    std::string nombre;
    double temperatura;
    bool activo;
};
```

Las clases también permiten definir tipos propios y se estudiarán con mayor profundidad en POO.

Ejemplos progresivos

Nivel 1 — Tipos básicos

```
#include <iostream>
#include <string>

int main() {
    int edad = 35;
    double altura = 1.75;
    char inicial = 'F';
    bool programador = true;
    std::string nombre = "Francisco";

    std::cout << nombre << std::endl;
    std::cout << edad << std::endl;
```

```

        std::cout << altura << std::endl;
        std::cout << inicial << std::endl;
        std::cout << programador << std::endl;

        return 0;
    }

```

La edad es entera, la altura puede tener fracción, inicial es un carácter, programador es lógico y nombre es texto. Error común: usar comillas simples para texto completo.

Nivel 2 — Tipos + entrada

```

#include <iostream>
#include <string>

int main() {
    std::string nombre;
    int piezas;
    double temperatura;

    std::cout << "Nombre: ";
    std::cin >> nombre;

    std::cout << "Piezas: ";
    std::cin >> piezas;

    std::cout << "Temperatura: ";
    std::cin >> temperatura;

    std::cout << "Datos recibidos." << std::endl;
    return 0;
}

```

Los tipos reflejan la naturaleza del dato. Si temperatura fuera int, los decimales no podrían representarse de la misma forma.

Nivel 3 — Aplicación industrial

```

#include <iostream>
#include <string>

int main() {
    std::string maquina = "Torno CNC";
    std::string modelo = "SE34";
    double temperatura = 82.5;
    int piezas = 450;
    bool activa = true;

    const double limite = 80.0;
    bool alerta = temperatura > limite;

    std::cout << "Máquina: " << maquina << std::endl;
    std::cout << "Modelo: " << modelo << std::endl;
    std::cout << "Temperatura: " << temperatura << " C" << std::endl;
    std::cout << "Piezas: " << piezas << std::endl;

    if (alerta)
        std::cout << "ALERTA" << std::endl;

    return 0;
}

```

Cada tipo corresponde al dominio. El límite es constante y alerta es booleano. Error común: usar varios booleanos para estados que realmente tienen tres o más posibilidades.

Nivel 4 — Modelado profesional

```

#include <iostream>
#include <string>

enum class EstadoMaquina {
    Detenida,
    Produciendo,
    Mantenimiento
}

```

```

};

struct Maquina {
    std::string codigo;
    std::string modelo;
    double temperatura;
    int piezasTurno;
    EstadoMaquina estado;
};

int main() {
    const double LIMITE_TEMPERATURA = 80.0;

    Maquina maquina{
        "MC-01", "SE34", 78.4, 525,
        EstadoMaquina::Produciendo
    };

    bool alerta = maquina.temperatura > LIMITE_TEMPERATURA;

    std::cout << "Código: " << maquina.codigo << std::endl;
    std::cout << "Modelo: " << maquina.modelo << std::endl;
    std::cout << "Temperatura: " << maquina.temperatura << std::endl;
    std::cout << "Piezas: " << maquina.piezasTurno << std::endl;
    std::cout << "Alerta: " << alerta << std::endl;

    return 0;
}

```

Aquí el programa crea un modelo de dominio: Maquina agrupa datos y EstadoMaquina expresa estados posibles. La alerta resulta false porque 78.4 no supera 80.0.

Diagramas y representaciones visuales

```

Dato real
|
v
Clasificar
|
+--> Entero -----> int
+--> Decimal -----> double
+--> Lógico -----> bool
+--> Carácter -----> char
+--> Texto -----> std::string
+--> Estados -----> enum class
+--> Entidad -----> struct/class

Elegir tipo
|
+-- ¿Texto? ----- sí -> string
|
+-- ¿true/false? --- sí -> bool
|
+-- ¿Entero? ----- sí -> int / variante
|
+-- ¿Decimal? ----- sí -> double / requisito específico
|
+-- ¿Estados? ----- sí -> enum class
|
+-- ¿Varios datos? - sí -> struct/class

```

Comparaciones importantes

Concepto	Qué es	Cuándo usarlo	Diferencia
int	Entero	Cantidades generales	No representa fracciones
unsigned	Entero no negativo	Dominio no negativo con conversión de tipos	Puede ser peligroso al mezclar signos
float	Punto flotante	Requisito específico de memoria/precisión	Menor precisión usual

Concepto	Qué es	Cuándo usarlo	Diferencia
double	Punto flotante	Cálculo general	No todos los decimales son exactos
char	Carácter	Un carácter	No es texto completo
bool	Lógico	Dos estados	No modela varios estados
std::string	Texto	Nombres/mensajes	Es una clase
enum class	Estados nombrados	Conjunto cerrado	Más expresivo que varios bool
struct	Tipo compuesto	Agrupar datos	Modela entidades
auto	Deducción	Evitar repetir tipos complejos	Sigue siendo tipado estático

Errores de confusión comunes

- 'A' es char; "A" es string.
- 10 es int; 10.5 es un literal de punto flotante.
- bool sirve para dos estados; enum class puede representar varios estados definidos.
- sizeof indica bytes ocupados; no es el valor máximo.
- auto deduce un tipo; no significa tipado dinámico.
- const es una propiedad de un objeto, no un tipo fundamental separado.

Errores comunes al trabajar con Tipos de Datos en C++

- **Usar int para todo:** puede eliminar fracciones o representar mal el dominio. Analiza el dato antes de elegir.
- **Confundir char y string:** carácter individual frente a texto de longitud variable.
- **Ignorar el rango:** un contador puede superar el límite de un tipo. Consulta límites cuando importe.
- **Asumir tamaños universales:** usa sizeof y propiedades estándar cuando la plataforma sea relevante.
- **Convertir double a int sin pensar:** se pierde la fracción.
- **Mezclar signed y unsigned:** las conversiones pueden producir resultados inesperados.
- **Comparar floats directamente:** considera que algunos valores no son representables exactamente.
- **Usar bool para demasiados estados:** considera enum class.
- **Usar auto sin entender la deducción:** recuerda que sigue existiendo un tipo concreto.
- **Usar un tipo por costumbre:** la elección debe responder al significado, rango y precisión.

Buenas prácticas

- Elige por significado del dato, no por hábito.
- Usa int como opción general para enteros cuando su rango sea suficiente.
- Usa double como punto de partida para punto flotante general, salvo requisitos específicos.
- Usa bool para estados binarios claros.
- Usa enum class para conjuntos cerrados de estados.
- Usa const para límites y valores que no deben cambiar.

- Prefiere `static_cast` para conversiones explícitas intencionales.
- Consulta `numeric_limits` cuando los límites importen.
- Consulta `sizeof` cuando el almacenamiento importe.
- Evita conversiones implícitas que oculten pérdida de información.
- Usa `std::string` para texto normal.
- En sistemas de bajo nivel, documenta supuestos de plataforma y representación.
- Prueba valores normales y valores cercanos a límites.
- Define explícitamente los requisitos de precisión en cálculos sensibles.

Conectando conocimientos

Tema anterior	Relación con tipos
Variables	Cada variable necesita un tipo.
Entradas con <code>cin</code>	El tipo determina qué dato espera el programa.
Salidas con <code>cout</code>	El valor se muestra según su tipo.
Comentarios	Documentan unidades, supuestos y decisiones.
Operadores	Las operaciones y conversiones dependen de los tipos.
Condiciones	Las expresiones booleanas participan en decisiones.
Arrays	Todos sus elementos tienen un tipo.
Funciones	Parámetros y valores de retorno tienen tipos.
POO	<code>struct/class</code> permiten construir tipos propios.

Ejercicios prácticos

No se proporcionan soluciones. Resuelve primero y después revisa tu diseño.

Ejercicio 1 — Comprensión básica

Objetivo: reconocer tipos. **Enunciado:** declara variables para edad, temperatura, inicial, máquina activa y nombre. **Requisitos:** usa un tipo adecuado para cada dato y muéstralos. **Pista:** clasifica primero cada dato.

Ejercicio 2 — Aplicación

Objetivo: seleccionar tipos. **Contexto:** tienda con producto, precio, cantidad y descuento. **Restricciones:** no uses `int` para valores que necesiten fracciones. **Pistas:** analiza signo, rango y precisión.

Ejercicio 3 — Integración

Objetivo: combinar tipos, entradas, salidas y decisiones. **Enunciado:** registra temperatura y piezas y determina alerta. **Requisitos:** usa `double`, `int`, `bool` y `const` cuando corresponda. **Pista:** el tipo debe representar la naturaleza del dato.

Ejercicio 4 — Caso práctico

Objetivo: modelar una estación de producción. Registra código, modelo, temperatura, velocidad, piezas y estado. Decide el tipo de cada dato. **Restricción:** evita múltiples `bool` si existe un conjunto cerrado de estados. **Pista:** considera `enum class`.

Ejercicio 5 — Reto

Problema: diseña el modelo de datos de cinco máquinas. **Requisitos:** identidad, producción, temperatura, velocidad, estado, alarma y mantenimiento. **Reglas:** considera rango, precisión, claridad y crecimiento. **Funcionalidades:** registrar, resumir, detectar alertas y contar producción. **Pistas:** clasifica, analiza límites y después decide si necesitas struct y enum class.

Mini proyecto del capítulo

Nombre

Sistema de monitoreo tipado de una estación de producción

Contexto y objetivo

Una empresa necesita una aplicación de consola que represente una máquina y su información principal. El propósito es practicar la selección de tipos y el modelado de datos antes de avanzar a programas más grandes.

Requisitos funcionales

- Registrar código y modelo.
- Registrar temperatura, velocidad y piezas.
- Representar el estado.
- Determinar alerta de temperatura.
- Mostrar resumen.
- Preparar el diseño para varias máquinas.

Requisitos técnicos

- Usar string, int, double y bool donde corresponda.
- Usar const para límites.
- Usar enum class para estados.
- Usar struct para agrupar datos.
- Evitar conversiones innecesarias.
- Considerar rango y precisión.

Paso 1

Haz una tabla de datos: nombre, significado, tipo propuesto, rango esperado y motivo.

Paso 2

Implementa el registro mediante cin y verifica que cada entrada corresponda al tipo elegido.

Paso 3

Agrega el límite de temperatura y la condición de alerta.

Paso 4

Agrupar los datos de una máquina en un tipo compuesto. Decide qué campos pertenecen al modelo.

Paso final

Adapta tu diseño para cinco máquinas sin perder claridad. Revisa conversiones, límites y estados.

Pistas: modela primero el dominio; después escribe el código. Un buen tipo puede prevenir errores que de otro modo aparecerían más adelante.

Comprueba tus conocimientos

1. ¿Qué es un tipo de dato?
2. ¿Por qué C++ necesita tipos?
3. ¿Cuál es la diferencia entre `int` y `double`?
4. ¿Para qué sirve `bool`?
5. ¿Cuál es la diferencia entre `char` y `std::string`?
6. ¿Qué indica `sizeof`?
7. ¿Por qué no debes asumir un tamaño universal para `int`?
8. ¿Qué es una conversión implícita?
9. ¿Qué ocurre al convertir 19.99 de `double` a `int`?
10. ¿Qué significa `const`?
11. ¿`auto` elimina el tipado estático?
12. ¿Cuándo usarías `enum class`?
13. ¿Qué riesgo existe al mezclar `signed` y `unsigned`?
14. ¿Qué diferencia hay entre rango y precisión?
15. ¿Qué problema existe en `int temperatura = 25.7;`?
16. ¿Por qué `char nombre = 'Luis';` es incorrecto?
17. ¿Qué alternativa considerarías para tres estados de una máquina?
18. ¿Cómo mejora un tipo correcto el mantenimiento?

Respuestas y explicaciones

1. Es una clasificación que determina qué clase de valor representa un objeto y qué reglas se aplican.
2. Permite representar distintas clases de información, organizar memoria y ayudar al compilador a detectar usos incompatibles.
3. `int` representa enteros; `double` representa punto flotante.
4. Representa `true` o `false` y es apropiado para condiciones binarias.
5. `char` representa un carácter; `std::string` representa texto de longitud variable.
6. Los bytes ocupados por un tipo u objeto en la implementación concreta.
7. Porque C++ admite implementaciones con características distintas; consulta cuando el tamaño importe.
8. Es una conversión realizada automáticamente por las reglas del lenguaje.
9. La parte fraccionaria se pierde; el resultado entero es 19.
10. Que el objeto no puede modificarse a través de ese nombre después de inicializarlo.
11. No. auto deduce un tipo concreto en compilación.
12. Cuando existe un conjunto cerrado de estados nombrados.
13. Comparaciones y conversiones pueden producir resultados inesperados.
14. Rango es el conjunto de valores representables; precisión es el detalle significativo que puede conservar una representación.
15. Se intenta guardar un decimal en un entero y se pierde la parte fraccionaria.
16. `char` es para un carácter individual, no para varias letras.
17. `enum class` es una opción clara para estados mutuamente excluyentes.
18. Hace explícita la intención, reduce errores y facilita comprender y modificar el modelo.

Resumen del capítulo

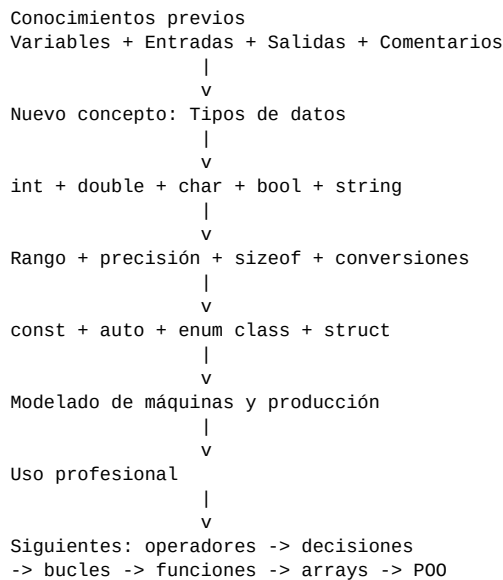
Concepto	Para qué sirve	Cuándo utilizarlo	Error común
<code>int</code>	Enteros	Cantidades	Ignorar rango
<code>double</code>	Punto flotante	Mediciones/cálculos	Esperar exactitud decimal total
<code>bool</code>	Lógico	Dos estados	Representar muchos estados
<code>char</code>	Carácter	Un carácter	Confundir con <code>string</code>
<code>std::string</code>	Texto	Nombres/mensajes	Usar <code>char</code> para texto
<code>const</code>	Evitar modificaciones	Límites	Tratarlo como tipo independiente
<code>enum class</code>	Estados	Opciones cerradas	Usar <code>bool</code> ambiguos
<code>struct</code>	Agrupar datos	Entidades	Separar datos relacionados
<code>static_cast</code>	Conversión explícita	Conversión intencional	Ignorar pérdida
<code>sizeof</code>	Consultar tamaño	Memoria/portabilidad	Asumir tamaño universal

Ideas clave que debes recordar

- El tipo describe la clase de valor.
- Elige por significado, rango y precisión.

- int es para enteros y double para punto flotante general.
- bool representa verdadero/falso.
- char y string son diferentes.
- sizeof consulta almacenamiento.
- numeric_limits consulta límites.
- Las conversiones pueden perder información.
- const expresa invariantes.
- enum class modela estados cerrados.
- struct agrupa datos relacionados.
- auto deduce un tipo, no elimina el tipado estático.

Mapa de aprendizaje



Este capítulo prepara directamente los siguientes temas: los operadores trabajan sobre valores tipados, las decisiones utilizan expresiones booleanas, los arrays contienen elementos de un tipo y las funciones reciben y devuelven valores con tipos definidos.

Verificación final

Pregunta	Respuesta dentro del capítulo
¿Qué es?	Fundamentos y tipos.
¿Por qué existe?	Introducción y representación.
¿Para qué sirve?	Selección y modelado.
¿Cómo funciona?	Bits, memoria, rango, precisión y conversiones.
¿Cuándo se utiliza?	Criterios de selección y ejemplos.
¿Cómo se aplica?	Ejemplos, ejercicios y proyecto.
¿Errores comunes?	Sección específica.

Pregunta	Respuesta dentro del capítulo
¿Buenas prácticas?	Sección profesional.
¿Cómo se practica?	Cinco ejercicios y mini proyecto.
¿Uso real?	Monitoreo de máquinas y producción.